

PYTHON FUNDAMENTALS

Session 1.2: Working with Strings

Instructor: Lance

STRUCTURE OF THE DAY

Week 1



Part 1

Strings: Text
Basics, indexing
and methods.



Part 2

Workshop: Text
Processing
Practice.



Part 3

Git Workflow:
Daily Professional
Practice.



Part 4

Workshop: Git
Mastery &
Commit Cycle.

PART 1: Strings

Working with Text

LEARNING OBJECTIVES

Strings: Create and manipulate text data in Python.

Indexing: Access individual characters and slices.

Methods: Use built-in tools: strip, upper, split, join.

F-Strings: Format output professionally.

Git: Make the daily commit workflow automatic.

WHAT ARE STRINGS?

A string is any text in Python — names, emails, messages, and data labels.

Names

```
'Sarah Johnson'
```

Emails

```
'user@email.com'
```

Messages

```
'Hello, World!'
```

Labels

```
'positive',  
'negative'
```

In ML and Data Science, strings are everywhere: text classification, sentiment analysis, NLP.

CREATING STRINGS

Three Ways to Create Strings

1. Single quotes: 'Hello'
2. Double quotes: "Hello"
3. Triple quotes: """Hello"""

Use double when string has an apostrophe:

"It's a beautiful day"

Triple quotes span multiple lines.

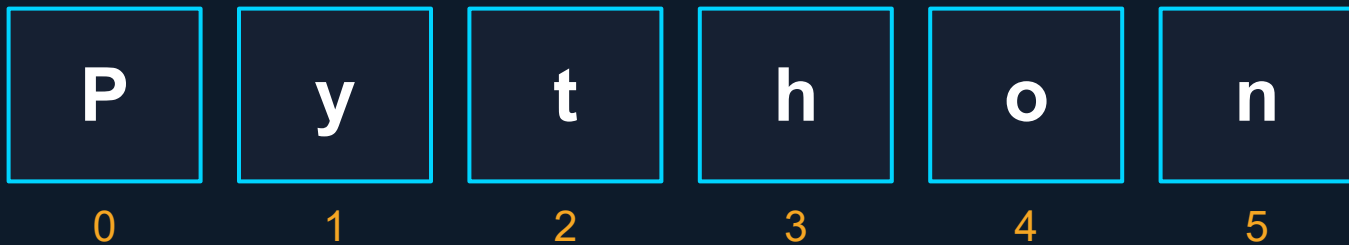
```
# Single quotes
name = 'Alice'

# Double quotes (apostrophe)
msg = "It's Monday!"

# Triple quotes (multi-line)
para = '''
    Long description here.
    Spans many lines.
'''
```

STRINGS ARE SEQUENCES

Think of strings like beads on a necklace — each character has a position (index).



← Index numbers (positions)

Python counts from 0, not 1! This is called **zero-indexing** — get used to it!

```
word = 'Python'
print(word[0])    # P          print(word[-1])  # n   (last char)
print(word[1])    # y          print(word[-2])  # o   (second to last)
```

STRING SLICING

Slicing = Getting a Portion

Format: `string[start:stop]`

Rules:

- start — included
- stop — NOT included
- Missing start → from beginning
- Missing stop → to the end

Like cutting a piece of ribbon:
start at X, cut before Y.

```
text = 'Hello World'

# First 5 chars
text[0:5]    # 'Hello'

# From position 6 to end
text[6:]     # 'World'

# Shortcut — skip the 0
text[:5]     # 'Hello'

# Last 5 characters
text[-5:]    # 'World'
```

REAL-WORLD SLICING

Practical example: processing email addresses the way real data pipelines do.

```
email = 'sarah.johnson@techcompany.com'

# Find position of @
at_position = email.index('@')    # Result: 13

# Extract username (before @)
username = email[:at_position]
print(username)                  # 'sarah.johnson'

# Extract domain (after @)
domain = email[at_position+1:]
print(domain)                    # 'techcompany.com'
```

1 Extract username

2 Extract domain

3 Get file extension

4 Parse timestamps

STRING METHODS

Python has built-in tools for strings — like Excel functions, but for text.

`.upper()` → MAKE ALL UPPERCASE

`.lower()` → make all lowercase

`.strip()` → Remove spaces from ends

`.replace()` → Replace text in string

`.split()` → Cut string into a list

Format: `string.method()` Example: `name.upper()`

STRING METHODS IN ACTION

```
name = ' sarah johnson '  
  
# Remove spaces from ends  
clean = name.strip()  
print(clean)           # 'sarah johnson'  
  
# Make uppercase  
upper = clean.upper()  
print(upper)           # 'SARAH JOHNSON'  
  
# Title case (First Letter Capital)  
title = clean.title()  
print(title)           # 'Sarah Johnson'  
  
# Replace text  
new = clean.replace('sarah', 'Sarah')  
print(new)             # 'Sarah johnson'
```

F-STRINGS: MODERN PYTHON MAGIC

Why F-Strings?

Format: `f'text {variable}'`

- Easy to read
- Fast to write
- Professional standard
- Industry best practice

Used in ML for:

Logging training progress,
reporting results, creating
filenames, showing metrics.

```
name = 'Sarah'  
age = 32
```

```
# Old way - DON'T use  
msg = 'Name: ' + name
```

```
# F-string - USE THIS  
msg = f'Name: {name}'
```

```
# Math inside braces  
score = 87  
print(f'{score}% = {score/100:.2f}')
```

87% = 0.87

SPLIT AND JOIN

```
sentence = 'I love learning Python for ML'

# Split into words (creates a list)
words = sentence.split()
print(words)      # ['I', 'love', 'learning', 'Python', 'for', 'ML']

# Count words
word_count = len(words)
print(f'Word count: {word_count}')    # 6

# Join words back together
rejoined = ' '.join(words)
print(rejoined)    # 'I love learning Python for ML'
```

.startswith() Check if string starts with value

.endswith() Check if string ends with value

'text' in s Check if string contains value

DATA CLEANING IN PRACTICE

```
# Messy data from spreadsheet
raw_data = [' JOHN SMITH ', ' alice JOHNSON', 'BOB jones']

# Clean it up!
clean_data = []
for name in raw_data:
    clean = name.strip().title()
    clean_data.append(clean)

print(clean_data)
# ['John Smith', 'Alice Johnson', 'Bob Jones']

# Alternative — handle multiple internal spaces:
for name in raw_data:
    clean = " ".join(name.split()).title()
    clean_data.append(clean)
```

Companies pay good money for this skill — real data is always messy!

PART 2: Workshop

Text Processing Practice

EX 1: NAME FORMATTER

Task

File: name_formatter.py

```
name1 = 'john smith'  
name2 = ' ALICE JOHNSON '  
name3 = 'bob JONES'
```

Steps

1. Remove extra spaces with `.strip()`
2. Convert to title case with `.title()`
3. Print formatted names

Expected Output

```
John Smith  
Alice Johnson  
Bob Jones
```

Bonus: Use f-strings to label each result professionally.

EX 2: EMAIL VALIDATOR

Task

File: email_validator.py

```
email1 = 'user@company.com'  
email2 = 'invalid.email'  
email3 = ' USER@EMAIL.COM '
```

For each email:

1. Remove spaces (.strip())
2. Lowercase (.lower())
3. Check for '@' and '.'
4. Print 'Valid' or 'Invalid'

Bonus: Extract username & domain from valid emails.

```
email = email3.strip().lower()  
  
if '@' in email and '.' in email:  
    print('Valid')  
    at = email.index('@')  
    user = email[:at]  
    domain = email[at+1:]  
    print(f'User: {user}')  
    print(f'Domain: {domain}')  
else:  
    print('Invalid')
```

EX 3: TEXT STATISTICS TOOL

File: `text_stats.py` | `text = 'I am learning Python and I love it!'`

1. Total characters (including spaces) → `len(text)`
2. Total characters (no spaces) → `len(text.replace(' ', ''))`
3. Number of words → `len(text.split())`
4. Average word length → `total_chars / word_count`
5. Uppercase version → `text.upper()`
6. Lowercase version → `text.lower()`

This is text analytics — the foundation of Natural Language Processing!



10 MIN BREAK

Stretch, hydrate, and prepare for Git workflow.

PART 3: Git Workflow

Daily Professional Practice

THE THREE-STEP GIT CYCLE



Goal: Git becomes muscle memory — as natural as Ctrl+S.

Commit 3–10 times per day!

WHEN TO COMMIT?

✓ DO Commit When:

- You finish a feature (even a small one!)
- Code works / passes a test
- Before trying something risky
- End of work session
- Before taking a break

✗ DON'T Commit When:

- Code has errors (fix them first!)
- Half-way through a feature
- You haven't tested it yet

Rule of thumb: "Would I want to come back to this version?" — If yes, commit!

WRITING GOOD COMMIT MESSAGES

Good

```
'Add email validation function'
```

```
'Fix bug in name formatting'
```

```
'Update text cleaning logic'
```

```
'Complete Exercise 2 from Session 1.2'
```

Bad

```
'update'
```

```
'changes'
```

```
'stuff'
```

```
'fixed it'
```

Good messages = professional portfolio!

GIT STATUS & GIT DIFF

git status — Your Best Friend

ALWAYS use before any git command.

It tells you:

- What branch you're on
- What files changed
- What's staged (ready)
- What's not staged
- What's new (untracked)

git diff — See What Changed

Shows you exactly what changed:

- + Lines you added (green)
 - Lines you removed (red)
- Context around changes

Professional habit:

```
git diff →  
git add →  
git commit
```

VIEWING YOUR HISTORY

```
# See all commits
```

```
git log
```

```
# Easier to read – one line per commit
```

```
git log --oneline
```

```
# Output example:
```

```
# a1b2c3d  Add email validator
```

```
# e4f5g6h  Complete text exercises
```

```
# i7j8k9l  Add name formatter
```

```
# See last 5 commits only
```

```
git log -5
```

```
# This is your work timeline – proves you did the work (portfolio!)
```

GIT BEST PRACTICES

- 1 `git status` before EVERY command. (Seriously, every single time.)
- 2 Review with `git diff` before committing. (Know what you're saving.)
- 3 Write clear commit messages. (Future you will thank you.)
- 4 Commit often — 3–10 times per day. (Small, focused commits.)
- 5 Never commit broken code. (Test first!)

PART 4: Workshop

Git Mastery Practice

EX 4: COMPLETE GIT CYCLE

Step 1 — Setup

Create folder:
session_1_2_work

cd into the folder

git init

Step 2 — Save Exercises

Add name_formatter.py
git add + git commit

Add email_validator.py
git add + git commit

Add text_stats.py
git add + git commit

Step 3 — Make Changes

Edit one file
(add more features)

git diff
(see changes)

git add + git commit

git log --oneline

Goal: Git becomes automatic!

SESSION 1.2 SUMMARY

- ✓ Strings: Creation, indexing, and slicing.
- ✓ Methods: upper, lower, strip, split, join.
- ✓ F-Strings: Professional output formatting.
- ✓ Data Cleaning: Handling real messy data.
- ✓ Git: Daily commit workflow and best practices.

HOMework ASSIGNMENT

- 1 Complete the Session 1.2 Quiz (required).
- 2 Project: Build a Text Analyzer (`text_analyzer.py`) — clean input, report statistics, find words, format with f-strings.
- 3 Git Practice: Save your analyzer, make 3 improvements, commit after each, review with git log.
- 4 Prepare for Week 2: Read about lists — the next big topic!

LOOKING AHEAD: WEEK 2 — LISTS!

What Are Lists?

- Collections of values (like Excel columns)
- Store multiple items together
- Foundation for all data science work
- Indexing and slicing (just like strings!)

Why They Matter

ML works with collections of data, not single values.

Lists → NumPy arrays
→ Tensors → ML models

You can't do ML without mastering lists.

Get excited — lists unlock everything!